

Week-1

Name-Karthik kumar c

Exercise 1: Inventory Management System

Scenario:

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

Steps:

Understand the Problem:

Explain why data structures and algorithms are essential in handling large inventories.

Discuss the types of data structures suitable for this problem.

Setup:

Create a new project for the inventory management system.

Implementation:

Define a class Product with attributes like **productId**, **productName**, **quantity**, and **price**.

Choose an appropriate data structure to store the products (e.g., ArrayList, HashMap).

Implement methods to add, update, and delete products from the inventory.

Analysis:

Analyze the time complexity of each operation (add, update, delete) in your chosen data structure.

Discuss how you can optimize these operations.

Code:

```
import java.util.HashMap;
import java.util.Scanner;

class Product {
    int productId;
    String productName;
    int quantity;
    double price;

    Product(int productId, String productName, int
quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    public String toString() {
        return "ID: " + productId + ", Name: " +
productName +
        ", Quantity: " + quantity + ",
Price: " + price;
    }
}

public class InventoryManagementSystem {
    static HashMap<Integer, Product> inventory =
new HashMap<>();
    static Scanner sc = new Scanner(System.in);

    static void addProduct() {
        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Product Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Quantity: ");
        int quantity = sc.nextInt();
        System.out.print("Enter Price: ");
        double price = sc.nextDouble();
        inventory.put(id, new Product(id, name,
quantity, price));
        System.out.println("Product Added.");
    }
}
```

```

        static void updateProduct() {
            System.out.print("Enter Product ID to
Update: ");
            int id = sc.nextInt();

            if (inventory.containsKey(id)) {
                sc.nextLine();
                System.out.print("Enter New Product
Name: ");
                String name = sc.nextLine();

                System.out.print("Enter New Quantity:
");
                int quantity = sc.nextInt();

                System.out.print("Enter New Price: ");
                double price = sc.nextDouble();

                inventory.put(id, new Product(id, name,
quantity, price));
                System.out.println("Product Updated.");
            } else {
                System.out.println("Product Not
Found.");
            }
        }

        static void deleteProduct() {
            System.out.print("Enter Product ID to
Delete: ");
            int id = sc.nextInt();

            if (inventory.containsKey(id)) {
                inventory.remove(id);
                System.out.println("Product Deleted.");
            } else {
                System.out.println("Product Not
Found.");
            }
        }

        static void displayProducts() {
            if (inventory.isEmpty()) {

```

```

        System.out.println("Inventory is
Empty.");
    } else {
        for (Product p : inventory.values()) {
            System.out.println(p);
        }
    }
}

public static void main(String[] args) {
    int choice;

    do {
        System.out.println("\n1. Add Product");
        System.out.println("2. Update
Product");
        System.out.println("3. Delete
Product");
        System.out.println("4. Display
Products");
        System.out.println("5. Exit");
        System.out.print("Enter Choice: ");
        choice = sc.nextInt();
        switch (choice) {
            case 1:
                addProduct();
                break;
            case 2:
                updateProduct();
                break;
            case 3:
                deleteProduct();
                break;
            case 4:
                displayProducts();
                break;
            case 5:
                System.out.println("Exiting...");
                break;
            default:
                System.out.println("Invalid
Choice.");
        }
    }
}

```

```
        } while (choice != 5);  
    }  
}
```

Output:

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 1

Enter Product ID: 7

Enter Product Name: Iphone

Enter Quantity: 1

Enter Price: 100000

Product Added.

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 4

ID: 7, Name: Iphone, Quantity: 1, Price: 100000.0

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 2

Enter Product ID to Update: 7

Enter New Product Name: Motorola

Enter New Quantity: 2

Enter New Price: 11000

Product Updated.

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 1

Enter Product ID: 1

Enter Product Name: iphone

Enter Quantity: 1

Enter Price: 100000

Product Added.

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 4

ID: 1, Name: iphone, Quantity: 1, Price: 100000.0

ID: 7, Name: Motorola, Quantity: 2, Price: 11000.0

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 3

Enter Product ID to Delete: 7

Product Deleted.

Add Product

Update Product

Delete Product

Display Products

Exit

Enter Choice: 4

ID: 1, Name: iphone, Quantity: 1, Price: 100000.0

Add Product
Update Product
Delete Product
Display Products
Exit
Enter Choice: 5
Exiting...

Why are Data Structures and Algorithms important?

Data Structures and Algorithms play a key role in managing a large inventory system. They help organize product information efficiently, making it easier to store and retrieve data. They also reduce the time required to search, update, or delete product details, which improves the overall performance of the system.

Suitable Data Structures

An **ArrayList** can be used to store products in a sequence, but searching for a specific product takes more time because each element may need to be checked.

A **HashMap** is a better choice because it stores products using their Product ID as the key. This allows products to be added, updated, searched, and deleted much faster.

Time Complexity (Using HashMap)

Operation	Time Complexity
Add Product	$O(1)$
Update Product	$O(1)$
Delete Product	$O(1)$

Operation	Time Complexity
-----------	-----------------

Search Product	$O(1)$
----------------	--------

Optimization

The inventory system can be optimized by using a **HashMap** instead of an **ArrayList**. Since a HashMap accesses data through a unique key, most operations take constant time, making it suitable for handling large inventories efficiently. This results in faster performance and better scalability as the number of products increases.

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

Understand Asymptotic Notation:

Explain Big O notation and how it helps in analyzing algorithms.

Describe the best, average, and worst-case scenarios for search operations.

Setup:

Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

Implementation:

Implement linear search and binary search algorithms.

Store products in an array for linear search and a sorted array for binary search.

Analysis:

Compare the time complexity of linear and binary search algorithms.

Discuss which algorithm is more suitable for your platform and why

Code:

```
class SearchItem {
    int id;
    String name;
    String category;

    SearchItem(int id, String name, String category) {
        this.id = id;
        this.name = name;
        this.category = category;
    }
}

public class EcommerceSearch {

    static int findByLinear(SearchItem[] items, String productName) {

        for (int index = 0; index < items.length; index++) {
            if (items[index].name.equalsIgnoreCase(productName)) {
                return index;
            }
        }
        return -1;
    }

    static int findByBinary(SearchItem[] items, String productName) {

        int start = 0;
        int end = items.length - 1;

        while (start <= end) {

            int middle = start + (end - start) / 2;

            int value =
items[middle].name.compareToIgnoreCase(productName);

            if (value == 0) {
                return middle;
            }

            if (value < 0) {
                start = middle + 1;
            } else {
                end = middle - 1;
            }
        }

        return -1;
    }
}
```

```

static void display(SearchItem item) {
    System.out.println("Product ID : " + item.id);
    System.out.println("Product Name : " + item.name);
    System.out.println("Category : " + item.category);
}

public static void main(String[] args) {

    SearchItem[] products = {
        new SearchItem(101, "Laptop", "Electronics"),
        new SearchItem(102, "Mobile", "Electronics"),
        new SearchItem(103, "Shoes", "Fashion"),
        new SearchItem(104, "Watch", "Accessories")
    };

    String searchKey = "Shoes";

    int linearIndex = findByLinear(products, searchKey);

    System.out.println("Using Linear Search");
    if (linearIndex != -1) {
        display(products[linearIndex]);
    } else {
        System.out.println("Product not found.");
    }

    System.out.println();

    int binaryIndex = findByBinary(products, searchKey);

    System.out.println("Using Binary Search");
    if (binaryIndex != -1) {
        display(products[binaryIndex]);
    } else {
        System.out.println("Product not found.");
    }
}
}

```

Output

Using Linear Search

Product ID : 103

Product Name : Shoes

Category : Fashion

Using Binary Search

Product ID : 103

Product Name : Shoes

Category : Fashion

Explain Big O Notation

Answer:

Big O notation is used to measure the performance of an algorithm. It shows how much time an algorithm takes as the input size increases. It helps us compare different algorithms.

Best Case: The product is found in the first position.

Average Case: The product is found somewhere in the middle.

Worst Case: The product is found at the last position or is not available.

Product Class

Answer:

A SearchItem class is created with the following attributes:

id – Stores the product ID.

name – Stores the product name.

category – Stores the product category.

These details are used to search and identify products.

Compare the Time Complexity of Linear Search and Binary Search

Answer:

Algorithm	Time Complexity
Linear Search	$O(n)$
Binary Search	$O(\log n)$

Linear Search checks each element one by one. Binary Search divides the array into two parts and searches faster.

Which Algorithm is More Suitable and Why?

Answer:

Binary Search is better for large datasets because it is faster than Linear Search. It works only when the data is sorted. Linear Search is suitable for small or unsorted data.

Exercise 3: Sorting Customer Orders

Scenario:

You are tasked with sorting customer orders by their total price on an e-commerce platform. This helps in prioritizing high-value orders.

Steps:

Understand Sorting Algorithms:

Explain different sorting algorithms (Bubble Sort, Insertion Sort, Quick Sort, Merge Sort).

Setup:

Create a class **Order** with attributes like **orderId**, **customerName**, and **totalPrice**.

Implementation:

Implement **Bubble Sort** to sort orders by **totalPrice**.

Implement **Quick Sort** to sort orders by **totalPrice**.

Analysis:

Compare the performance (time complexity) of Bubble Sort and Quick Sort.

Discuss why Quick Sort is generally preferred over Bubble Sort.

Code:

```
class Order {  
    int orderId;
```

```

String customerName;
double totalPrice;

Order(int orderId, String customerName, double totalPrice) {
    this.orderId = orderId;
    this.customerName = customerName;
    this.totalPrice = totalPrice;
}
}

public class OrderSorting {

    static void bubbleSort(Order[] orders) {
        for (int i = 0; i < orders.length - 1; i++) {
            for (int j = 0; j < orders.length - i - 1; j++) {
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }

    static int partition(Order[] orders, int low, int high) {
        double pivot = orders[high].totalPrice;
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (orders[j].totalPrice < pivot) {
                i++;
                Order temp = orders[i];
                orders[i] = orders[j];
                orders[j] = temp;
            }
        }

        Order temp = orders[i + 1];
        orders[i + 1] = orders[high];
        orders[high] = temp;

        return i + 1;
    }

    static void quickSort(Order[] orders, int low, int high) {
        if (low < high) {
            int pivot = partition(orders, low, high);

            quickSort(orders, low, pivot - 1);
            quickSort(orders, pivot + 1, high);
        }
    }

    static void display(Order[] orders) {
        for (Order o : orders) {
            System.out.println(o.orderId + " " + o.customerName + " " +
o.totalPrice);
        }
    }

    public static void main(String[] args) {

```

```

        Order[] bubbleOrders = {
            new Order(101, "Karthik", 2500),
            new Order(102, "Rahul", 1200),
            new Order(103, "Anil", 3500),
            new Order(104, "Ravi", 1800)
        };

        Order[] quickOrders = {
            new Order(101, "Karthik", 2500),
            new Order(102, "Rahul", 1200),
            new Order(103, "Anil", 3500),
            new Order(104, "Ravi", 1800)
        };

        System.out.println("Bubble Sort");
        bubbleSort(bubbleOrders);
        display(bubbleOrders);

        System.out.println();

        System.out.println("Quick Sort");
        quickSort(quickOrders, 0, quickOrders.length - 1);
        display(quickOrders);
    }
}

```

Output:

Bubble Sort

102 Rahul 1200.0

104 Ravi 1800.0

101 Karthik 2500.0

103 Anil 3500.0

Quick Sort

102 Rahul 1200.0

104 Ravi 1800.0

101 Karthik 2500.0

103 Anil 3500.0

Explain Different Sorting Algorithms.

Answer:

Bubble Sort: Compares two adjacent elements and swaps them if they are in the wrong order.

Insertion Sort: Inserts each element into its correct position in the sorted part of the array.

Quick Sort: Selects a pivot element and divides the array into smaller parts before sorting.

Merge Sort: Splits the array into two halves, sorts them separately, and merges them back.

Exercise 4: Employee Management System**Scenario:**

You are developing an employee management system for a company. Efficiently managing employee records is crucial.

Steps:**1. Understand Array Representation:**

- Explain how arrays are represented in memory and their advantages.

2. Setup:

- Create a class **Employee** with attributes like **employeeId**, **name**, **position**, and **salary**.

3. Implementation:

- Use an array to store employee records.
- Implement methods to **add**, **search**, **traverse**, and **delete** employees in the array.

4. Analysis:

- Analyze the time complexity of each operation (add, search, traverse, delete).
- Discuss the limitations of arrays and when to use them.

Code

```
class Employee {
    int employeeId;
    String name;
    String position;
    double salary;

    Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }

    void display() {
        System.out.println(employeeId + " " + name + " " + position + " " +
salary);
    }
}

public class EmployeeManagement {

    static Employee[] employees = new Employee[5];
    static int count = 0;

    static void addEmployee(Employee emp) {
        if (count < employees.length) {
            employees[count] = emp;
            count++;
        } else {
            System.out.println("Array is Full");
        }
    }

    static void searchEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                System.out.println("Employee Found");
                employees[i].display();
                return;
            }
        }
        System.out.println("Employee Not Found");
    }

    static void traverseEmployees() {
        for (int i = 0; i < count; i++) {
            employees[i].display();
        }
    }

    static void deleteEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {

                for (int j = i; j < count - 1; j++) {
                    employees[j] = employees[j + 1];
                }
                count--;
            }
        }
    }
}
```

```

        }

        employees[count - 1] = null;
        count--;

        System.out.println("Employee Deleted");
        return;
    }
}

System.out.println("Employee Not Found");
}

public static void main(String[] args) {

    addEmployee(new Employee(101, "Karthik", "Developer", 50000));
    addEmployee(new Employee(102, "Rahul", "Tester", 45000));
    addEmployee(new Employee(103, "Anil", "Manager", 70000));

    System.out.println("Employee Records");
    traverseEmployees();

    System.out.println();

    searchEmployee(102);

    System.out.println();

    deleteEmployee(102);

    System.out.println();

    System.out.println("After Deletion");
    traverseEmployees();
}
}

```

Output:

Employee Records

101 Karthik Developer 50000.0

102 Rahul Tester 45000.0

103 Anil Manager 70000.0

Employee Found

102 Rahul Tester 45000.0

Employee Deleted

After Deletion

101 Karthik Developer 50000.0

103 Anil Manager 70000.0

1. Explain Array Representation.

Answer:

An array stores elements in continuous memory locations. Each element is accessed using an index. Arrays are easy to use and allow fast access to elements.

2. Employee Class

Answer:

The Employee class contains the following attributes:

- employeeId Stores the employee ID.
- name Stores the employee name.
- position Stores the employee's position.
- salary Stores the employee's salary.

These attributes are used to maintain employee records.

3. Analyze the Time Complexity of Add, Search, Traverse, and Delete.

Answer:

Operation Time Complexity

Add $O(1)$

Search $O(n)$

Traverse $O(n)$

Delete $O(n)$

4. Discuss the Limitations of Arrays and When to Use Them.

Answer:

Arrays have a fixed size, so they cannot grow or shrink after creation. Deleting or inserting elements takes more time because the remaining elements need to be shifted. Arrays are suitable when the number of elements is fixed and fast access using an index is required.

Exercise 5: Task Management System

Scenario:

You are developing a task management system where tasks need to be added, deleted, and traversed efficiently.

Steps:

1. Understand Linked Lists:

- Explain the different types of linked lists (Singly Linked List, Doubly Linked List).

2. Setup:

- Create a class **Task** with attributes like **taskId**, **taskName**, and **status**.

3. Implementation:

- Implement a singly linked list to manage tasks.

- Implement methods to **add**, **search**, **traverse**, and **delete** tasks in the linked list.

4. Analysis:

- Analyze the time complexity of each operation.
- Discuss the advantages of linked lists over arrays for dynamic data.

Code:

```
class Task {  
    int taskId;  
    String taskName;  
    String status;  
    Task next;  
  
    Task(int taskId, String taskName, String status) {  
        this.taskId = taskId;  
        this.taskName = taskName;  
        this.status = status;  
        this.next = null;  
    }  
}  
  
public class TaskManagement {  
  
    static Task head = null;
```

```
static void addTask(int id, String name, String status) {
```

```
    Task newTask = new Task(id, name, status);
```

```
    if (head == null) {
```

```
        head = newTask;
```

```
    } else {
```

```
        Task temp = head;
```

```
        while (temp.next != null) {
```

```
            temp = temp.next;
```

```
        }
```

```
        temp.next = newTask;
```

```
    }
```

```
}
```

```
static void searchTask(int id) {
```

```
    Task temp = head;
```

```
    while (temp != null) {
```

```
        if (temp.taskId == id) {
```

```
            System.out.println("Task Found");
```

```
            System.out.println(temp.taskId + " " + temp.taskName + " " +  
temp.status);
```

```
            return;
```

```
        }
```

```
        temp = temp.next;
```

```
}
```

```
System.out.println("Task Not Found");
```

```
}
```

```
static void traverseTasks() {
```

```
    Task temp = head;
```

```
    while (temp != null) {
```

```
        System.out.println(temp.taskId + " " + temp.taskName + " " +  
temp.status);
```

```
        temp = temp.next;
```

```
    }
```

```
}
```

```
static void deleteTask(int id) {
```

```
    if (head == null) {
```

```
        return;
```

```
    }
```

```
    if (head.taskId == id) {
```

```
        head = head.next;
```

```
        System.out.println("Task Deleted");
```

```
        return;
```

```
}
```

```
Task temp = head;
```

```
while (temp.next != null && temp.next.taskId != id) {
```

```
    temp = temp.next;
```

```
}
```

```
if (temp.next != null) {
```

```
    temp.next = temp.next.next;
```

```
    System.out.println("Task Deleted");
```

```
} else {
```

```
    System.out.println("Task Not Found");
```

```
}
```

```
}
```

```
public static void main(String[] args) {
```

```
    addTask(1, "Design", "Pending");
```

```
    addTask(2, "Coding", "In Progress");
```

```
    addTask(3, "Testing", "Completed");
```

```
    System.out.println("Task List");
```

```
    traverseTasks();
```

```
    System.out.println();
```

```
    searchTask(2);
```

```
    System.out.println();
```



```
        deleteTask(2);  
        System.out.println();  
        System.out.println("After Deletion");  
        traverseTasks();  
    }  
}
```

Output

Task List

1 Design Pending
2 Coding In Progress
3 Testing Completed

Task Found

2 Coding In Progress

Task Deleted

After Deletion

1 Design Pending
3 Testing Completed

1. Explain the Different Types of Linked Lists.

Answer:

Singly Linked List: Each node contains data and a link to the next node.

Doubly Linked List: Each node contains data, a link to the next node, and a link to the previous node.

2. Task Class

Answer:

The Task class contains the following attributes:

- taskId Stores the task ID.
- taskName Stores the task name.
- status Stores the current status of the task.
- next Stores the address of the next node.

3. Time Complexity

Operation Time Complexity

Add $O(n)$

Search $O(n)$

Traverse $O(n)$

Delete $O(n)$

4. Advantages of Linked Lists Over Arrays

Answer:

- Linked lists can grow and shrink dynamically.
- Insertion and deletion are easier than arrays.
- Memory is allocated only when needed.
- They are suitable when the size of data changes frequently.

Exercise 6: Library Management System

Scenario:

You are developing a library management system where users can search for books by title or author.

Steps:

1. Understand Search Algorithms:

- Explain linear search and binary search algorithms.

2. Setup:

- Create a class **Book** with attributes like **bookId**, **title**, and **author**.

3. Implementation:

- Implement linear search to find books by title.
- Implement binary search to find books by title (assuming the list is sorted).

4. Analysis:

- Compare the time complexity of linear and binary search.
- Discuss when to use each algorithm based on the data set size and order.

Code:

```
public class LibraryManagement{

    static int linearSearch(Book[] books,String key){
        for(int i=0;i<books.length;i++){
            if(books[i].title.equalsIgnoreCase(key)){
                return i;
            }
        }
        return -1;
    }

    static int binarySearch(Book[] books,String key){
```

```

        int low=0;
        int high=books.length-1;

        while(low<=high){
            int mid=(low+high)/2;

            int result=books[mid].title.compareToIgnoreCase(key);

            if(result==0){
                return mid;
            }
            else if(result<0){
                low=mid+1;
            }
            else{
                high=mid-1;
            }
        }
        return -1;
    }

    static void display(Book b){
        System.out.println("Book ID : "+b.bookId);
        System.out.println("Title : "+b.title);
        System.out.println("Author : "+b.author);
    }

    public static void main(String[] args){

        Book[] books={
            new Book(101,"C Programming","Dennis"),
            new Book(102,"Data Structures","Narasimha Karumanchi"),
            new Book(103,"Java","Herbert Schildt"),
            new Book(104,"Python","Mark Lutz")
        };

        String searchKey="Java";

        int linear=linearSearch(books,searchKey);

        System.out.println("Linear Search");
        if(linear!=-1){
            display(books[linear]);
        }
        else{
            System.out.println("Book Not Found");
        }

        System.out.println();

        int binary=binarySearch(books,searchKey);

        System.out.println("Binary Search");
        if(binary!=-1){
            display(books[binary]);
        }
        else{
            System.out.println("Book Not Found");
        }
    }
}

```

Output:

Linear Search

Book ID : 103

Title : Java

Author : Herbert Schildt

Binary Search

Book ID : 103

Title : Java

Author : Herbert Schildt

1. Explain Linear Search and Binary Search Algorithms.**Answer:**

Linear Search checks each element one by one until the required book is found.

Binary Search repeatedly divides the sorted array into two halves to find the required book. It works only on sorted data.

2. Book Class**Answer:**

The Book class contains the following attributes:

- bookId Stores the book ID.
- title Stores the book title.
- author Stores the author's name.

These attributes help identify and search for books.

3. Compare the Time Complexity of Linear Search and Binary Search.

Algorithm	Time Complexity
------------------	------------------------

Linear Search	$O(n)$
---------------	--------

Binary Search	$O(\log n)$
---------------	-------------

4. When to Use Each Algorithm?

Answer:

Linear Search is suitable for small or unsorted datasets. Binary Search is suitable for large datasets when the books are stored in sorted order because it searches much faster.

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- Explain the concept of recursion and how it can simplify certain problems.

2. Setup:

- Create a method to calculate the future value using a recursive approach.

3. Implementation:

- Implement a recursive algorithm to predict future values based on past growth rates.

4. Analysis:

- Discuss the time complexity of your recursive algorithm.
- Explain how to optimize the recursive solution to avoid excessive computation.

Code:

```
public class FinancialForecasting {  
  
    static double futureValue(double amount, double growthRate, int years) {  
  
        if (years == 0) {  
            return amount;  
        }  
  
        return futureValue(amount * (1 + growthRate), growthRate, years - 1);  
    }  
  
    public static void main(String[] args) {  
  
        double presentValue = 10000;  
        double growthRate = 0.10;  
        int years = 5;  
  
        double result = futureValue(presentValue, growthRate, years);  
  
        System.out.println("Present Value : " + presentValue);  
        System.out.println("Growth Rate : " + (growthRate * 100) + "%");  
        System.out.println("Years : " + years);  
        System.out.println("Future Value : " + result);  
    }  
}
```

Output:

Present Value : 10000.0

Growth Rate : 10.0%

Years : 5

Future Value : 16105.1

1. Explain the Concept of Recursion.

Answer:

Recursion is a technique in which a method calls itself to solve a problem. It continues calling itself until a base condition is reached. Recursion is useful for solving problems that can be divided into smaller subproblems.

2. Future Value Method

Answer:

The futureValue() method calculates the future amount using the present value, growth rate, and number of years. It uses recursion to calculate the value year by year.

3. Time Complexity of the Recursive Algorithm.

Operation	Time Complexity
------------------	------------------------

Recursive Future Value	$O(n)$
------------------------	--------

where **n** is the number of years.

4. How to Optimize the Recursive Solution?

Answer:

The recursive solution can be optimized by using memoization or an iterative approach. These methods reduce unnecessary recursive calls and improve performance, especially for large inputs.